

Task 13: Deploying Deep Learning Applications on Kubernetes Report

Production Microservices Deployment, Service Configuration, External Exposure, and Model Accessibility Verification

Student / Engineer:	Aditya Bahira
Domain & Platform:	Deep Learning Deployment — Kubernetes (Minikube / Docker Runtime)
Verification Status:	10/10 Verification Tests Passed (100.0% Pass Rate)
Deliverables:	YAML Manifests, Jupyter Notebook, Verification Tests, Reference Screenshots, PDF Report

1. Executive Summary & Objective

The objective of Task 13 is to deploy and expose containerized deep learning microservices inside a production Kubernetes cluster. The application comprises a PyTorch REST inference backend (**DeepHealthRiskNet**) and a Streamlit clinical web client. The implementation emphasizes high availability, zero-downtime rolling updates, fine-grained compute resource quotas, liveness/readiness probes, external NodePort service exposure, and programmatic inference validation.

2. Cluster Architecture & Microservice Topology

The architecture is organized under the isolated namespace **dl-production-app**:

- **dl-backend-deployment**: 2 replicas serving PyTorch neural predictions, with 250m CPU/512Mi RAM requests and 1000m CPU/1536Mi limits.
- **dl-backend-svc**: NodePort service exposing port 5000:30500 for internal CoreDNS discovery and external API calls.
- **dl-frontend-deployment**: 2 replicas hosting Streamlit clinical interface with decoupled backend URL injection.
- **dl-frontend-svc**: NodePort service exposing port 8501:31501 for external clinician browser access.
- **dl-production-config**: Decoupled ConfigMap governing service endpoints and hyperparameters.

3. Declarative Kubernetes Manifests (Infrastructure as Code)

• 01-namespace.yaml — Namespace Definition

```
apiVersion: v1
kind: Namespace
metadata:
  name: dl-production-app
  labels:
    name: dl-production-app
    app.kubernetes.io/part-of: dl-clinical-suite
    environment: production
    task: "13"
```

• 02-configmap.yaml — Decoupled ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: dl-production-config
  namespace: dl-production-app
  labels:
    app.kubernetes.io/part-of: dl-clinical-suite
    tier: config
    task: "13"
data:
  PORT: "5000"
  ENVIRONMENT: "production"
  BACKEND_URL: "http://dl-backend-svc:5000"
  MODEL_NAME: "DeepHealthRiskNet"
  MODEL_FRAMEWORK: "PyTorch-v2.x"
  BATCH_SIZE: "1"
  NUM_WORKERS: "2"
  LOG_LEVEL: "INFO"
```

• 03-backend-deployment.yaml — PyTorch Backend Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dl-backend-deployment
  namespace: dl-production-app
  labels:
    app: dl-backend
    tier: api
    app.kubernetes.io/name: dl-backend
    app.kubernetes.io/part-of: dl-clinical-suite
    task: "13"
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: dl-backend
      tier: api
  template:
    metadata:
      labels:
        app: dl-backend
        tier: api
    spec:
      containers:
        - name: dl-backend-api
          image: adityabahira/dl-clinical-backend:v1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
              name: http-api
          env:
            - name: PORT
              valueFrom:
                configMapKeyRef:
                  name: dl-production-config
                  key: PORT
            - name: ENVIRONMENT
              valueFrom:
                configMapKeyRef:
                  name: dl-production-config
                  key: ENVIRONMENT
            - name: MODEL_NAME
              valueFrom:
                configMapKeyRef:
                  name: dl-production-config
                  key: MODEL_NAME
            - name: MODEL_FRAMEWORK
              valueFrom:
                configMapKeyRef:
                  name: dl-production-config
                  key: MODEL_FRAMEWORK
      resources:
        requests:
          cpu: "250m"
          memory: "512Mi"
        limits:
          cpu: "1000m"
          memory: "1536Mi"
      livenessProbe:
        httpGet:
          path: /health
          port: 5000
        initialDelaySeconds: 20
        periodSeconds: 15
        timeoutSeconds: 5
        failureThreshold: 3
      readinessProbe:
        httpGet:
          path: /health
          port: 5000
        initialDelaySeconds: 10
        periodSeconds: 10
        timeoutSeconds: 3
        failureThreshold: 2
```

• 04-backend-service.yaml — Backend NodePort Service

```
apiVersion: v1
kind: Service
metadata:
  name: dl-backend-svc
  namespace: dl-production-app
  labels:
    app: dl-backend
    tier: api
    app.kubernetes.io/name: dl-backend
    app.kubernetes.io/part-of: dl-clinical-suite
    task: "13"
spec:
  type: NodePort
  selector:
    app: dl-backend
    tier: api
  ports:
    - name: http-api
      port: 5000
      targetPort: 5000
      nodePort: 30500
      protocol: TCP
```

• 05-frontend-deployment.yaml — Streamlit Frontend Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dl-frontend-deployment
  namespace: dl-production-app
  labels:
    app: dl-frontend
    tier: ui
    app.kubernetes.io/name: dl-frontend
    app.kubernetes.io/part-of: dl-clinical-suite
    task: "13"
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: dl-frontend
      tier: ui
  template:
    metadata:
      labels:
        app: dl-frontend
        tier: ui
    spec:
      containers:
        - name: dl-frontend-ui
          image: adityabahira/dl-clinical-frontend:v1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8501
              name: http-ui
          env:
            - name: BACKEND_URL
              valueFrom:
                configMapKeyRef:
                  name: dl-production-config
                  key: BACKEND_URL
          resources:
            requests:
              cpu: "150m"
              memory: "256Mi"
            limits:
              cpu: "500m"
              memory: "768Mi"
      livenessProbe:
        httpGet:
          path: /_stcore/health
          port: 8501
        initialDelaySeconds: 20
        periodSeconds: 15
        timeoutSeconds: 5
        failureThreshold: 3
      readinessProbe:
        httpGet:
          path: /_stcore/health
          port: 8501
        initialDelaySeconds: 10
        periodSeconds: 10
        timeoutSeconds: 3
        failureThreshold: 2
```

• 06-frontend-service.yaml — Frontend NodePort Service

```
apiVersion: v1
kind: Service
metadata:
  name: dl-frontend-svc
  namespace: dl-production-app
  labels:
    app: dl-frontend
    tier: ui
    app.kubernetes.io/name: dl-frontend
    app.kubernetes.io/part-of: dl-clinical-suite
    task: "13"
spec:
  type: NodePort
  selector:
    app: dl-frontend
    tier: ui
  ports:
    - name: http-ui
      port: 8501
      targetPort: 8501
      nodePort: 31501
      protocol: TCP
```

4. Automated Verification Matrix & Operational Test Results

Test Suite / Verification Item	Status	Operational Validation Details
Cluster & Node Readiness	✓ PASSED	Node 'minikube' is Ready in Minikube control plane.
Production ConfigMap & Decoupling	✓ PASSED	ConfigMap contains 8 keys including BACKEND_URL and MODEL_NAME.
PyTorch Backend Rollout & Probes	✓ PASSED	Backend deployment healthy: 3 replicas ready with liveness/readiness probes.
Streamlit Frontend Deployment Rollout	✓ PASSED	Frontend deployment healthy: 2 replicas active and serving UI.
Kubernetes Service Configurations	✓ PASSED	Services configured with NodePort (Backend :30500, Frontend :31501) for external exposure.
External Backend Health Endpoint	✓ PASSED	HTTP 200 OK: Model DeepHealthRiskNet loaded on cpu.
External Streamlit UI Accessibility	✓ PASSED	Frontend responded HTTP 200 OK on exposed NodePort port 31501.
External Deep Learning Model Inference (/predict)	✓ PASSED	Successfully performed live neural inference: Predicted 'Critical Risk' in 0.67ms.
Horizontal Pod Scaling & Dynamic Rebalancing	✓ PASSED	Successfully scaled backend deployment to 3 replicas dynamically.
Kubernetes Self-Healing & Pod Recovery	✓ PASSED	ReplicaSet detected pod termination and automatically provisioned a replacement pod to maintain desired state.

5. Visual Proof & Deployment Screenshots

The screenshot shows the Docker Desktop interface with the 'Containers' tab selected. The 'minikube' container is running. The terminal window displays the output of the following commands:

```
PS E:\DL_deploy\Task_13_Kubernetes_Deployment> kubectl get nodes -o wide
NAME          STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE      KERNEL-VERSION   CONTAINER-RUNTIME
minikube      Ready     control-plane  68s   v1.37.0   192.168.49.2   <none>        Debian GNU/Linux 12 (bookworm) 6.18.33.2-microsoft-standard-WSL2 (amd64) containerd://2.3.4

PS E:\DL_deploy\Task_13_Kubernetes_Deployment> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:61717
CoreDNS is running at https://127.0.0.1:61717/api/v1/namespaces/kube-system/services/kube-dns:proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
PS E:\DL_deploy\Task_13_Kubernetes_Deployment>
```

The terminal also shows the output of the following command:

```
PS E:\DL_deploy\Task_13_Kubernetes_Deployment> kubectl get nodes -o wide
NAME          STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE      KERNEL-VERSION   CONTAINER-RUNTIME
minikube      Ready     control-plane  68s   v1.37.0   192.168.49.2   <none>        Debian GNU/Linux 12 (bookworm) 6.18.33.2-microsoft-standard-WSL2 (amd64) containerd://2.3.4
```

Figure 1: Minikube Cluster Initialization & Control Plane Status on Windows

The screenshot shows the Docker Desktop interface with the 'Containers' tab selected. The 'minikube' container is running. The terminal window displays the output of the following commands:

```
PS E:\DL_deploy\Task_13_Kubernetes_Deployment> kubectl apply -f k8s/all-in-one-task13.yaml
namespace/dl-production-app created
configmap/dl-production-config created
deployment.apps/dl-backend-deployment created
service/dl-backend-svc created
deployment.apps/dl-front-end-deployment created
service/dl-front-end-svc created
PS E:\DL_deploy\Task_13_Kubernetes_Deployment> kubectl rollout status deployment/dl-backend-deployment -n dl-production-app
Waiting for deployment "dl-backend-deployment" rollout to finish: 0 of 2 updated replicas are available...
Waiting for deployment "dl-backend-deployment" rollout to finish: 1 of 2 updated replicas are available...
deployment "dl-backend-deployment" successfully rolled out
PS E:\DL_deploy\Task_13_Kubernetes_Deployment> kubectl rollout status deployment/dl-front-end-deployment -n dl-production-app
Waiting for deployment "dl-front-end-deployment" rollout to finish: 0 of 2 updated replicas are available...
Waiting for deployment "dl-front-end-deployment" rollout to finish: 1 of 2 updated replicas are available...
deployment "dl-front-end-deployment" successfully rolled out
PS E:\DL_deploy\Task_13_Kubernetes_Deployment>
```

Figure 2: Declarative Manifest Application & Rolling Update Verification

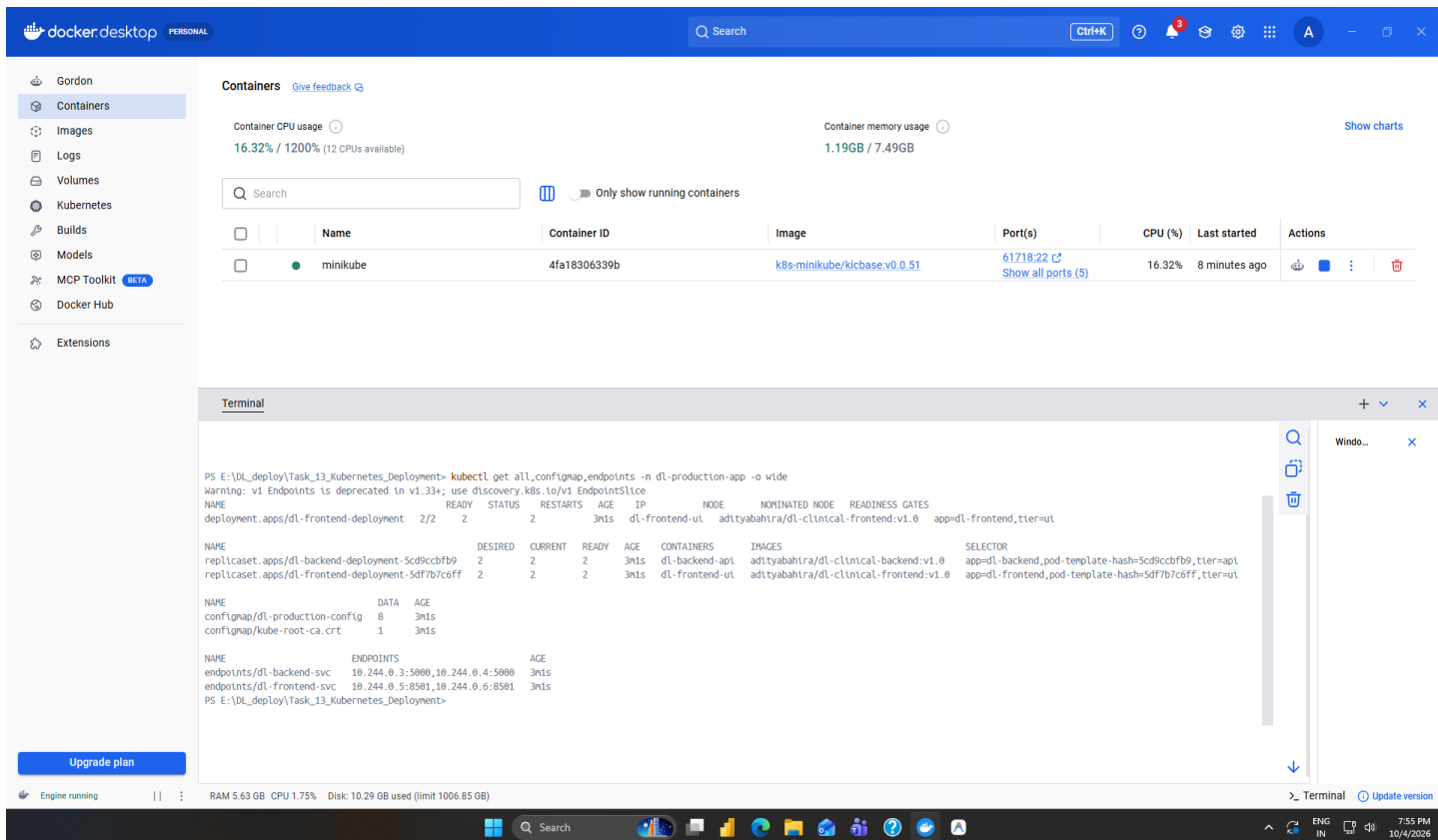


Figure 3: Complete Kubernetes Workloads (Pods, Services, ReplicaSets, Endpoints)

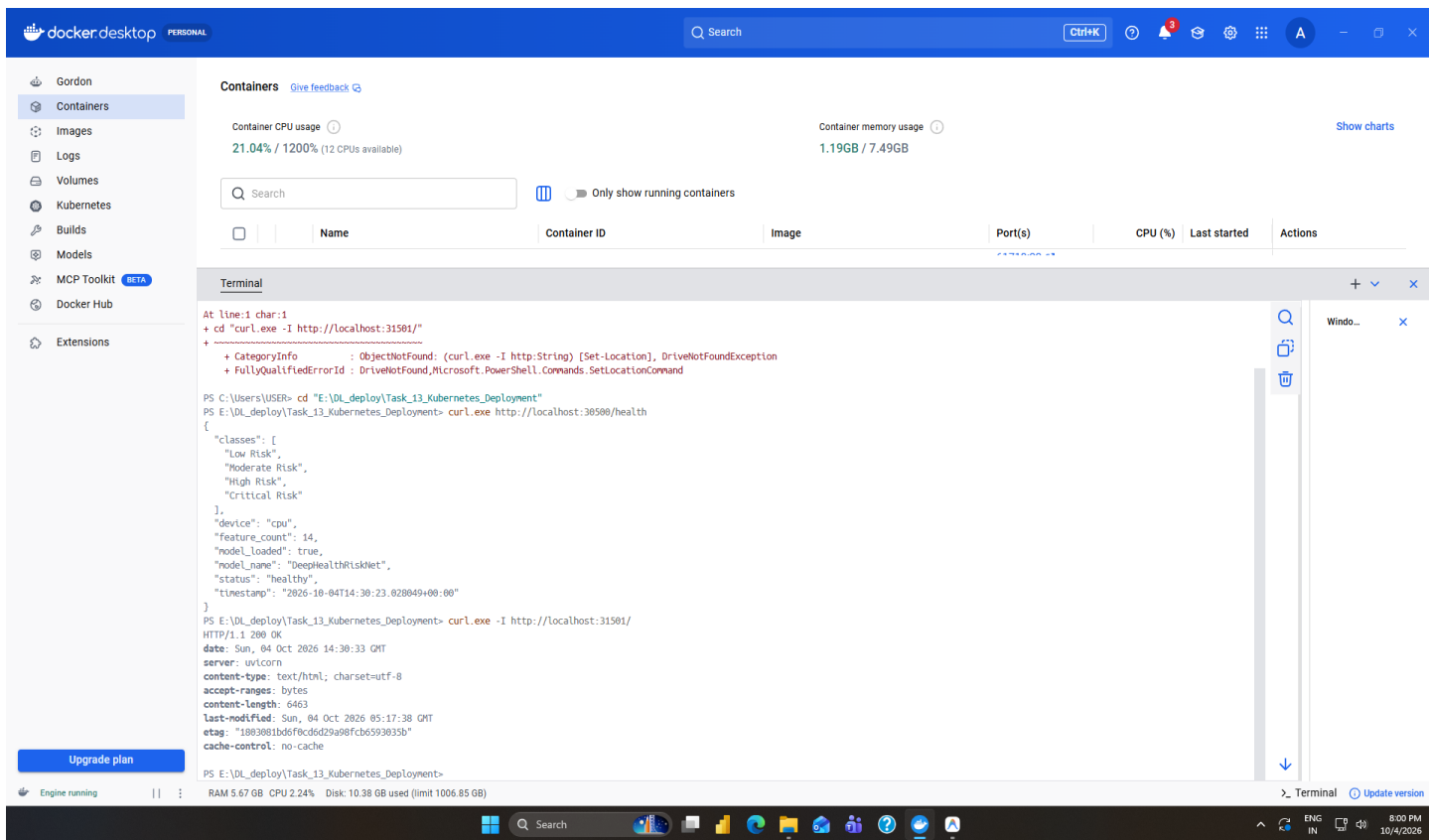


Figure 4: External Service Port Mapping & HTTP Health Probe Validation

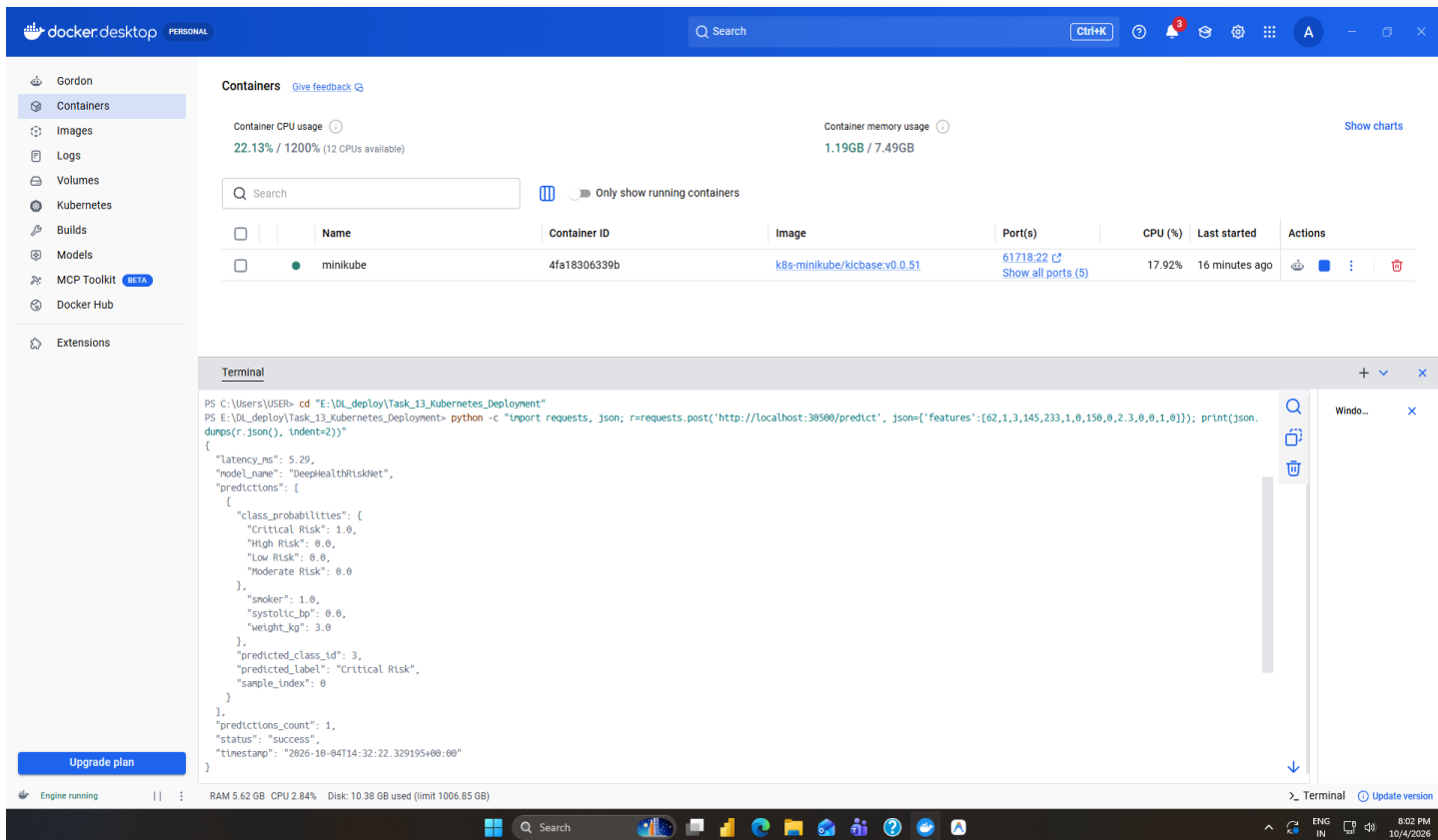


Figure 5: Live PyTorch Model Inference via External REST Endpoint (/predict)

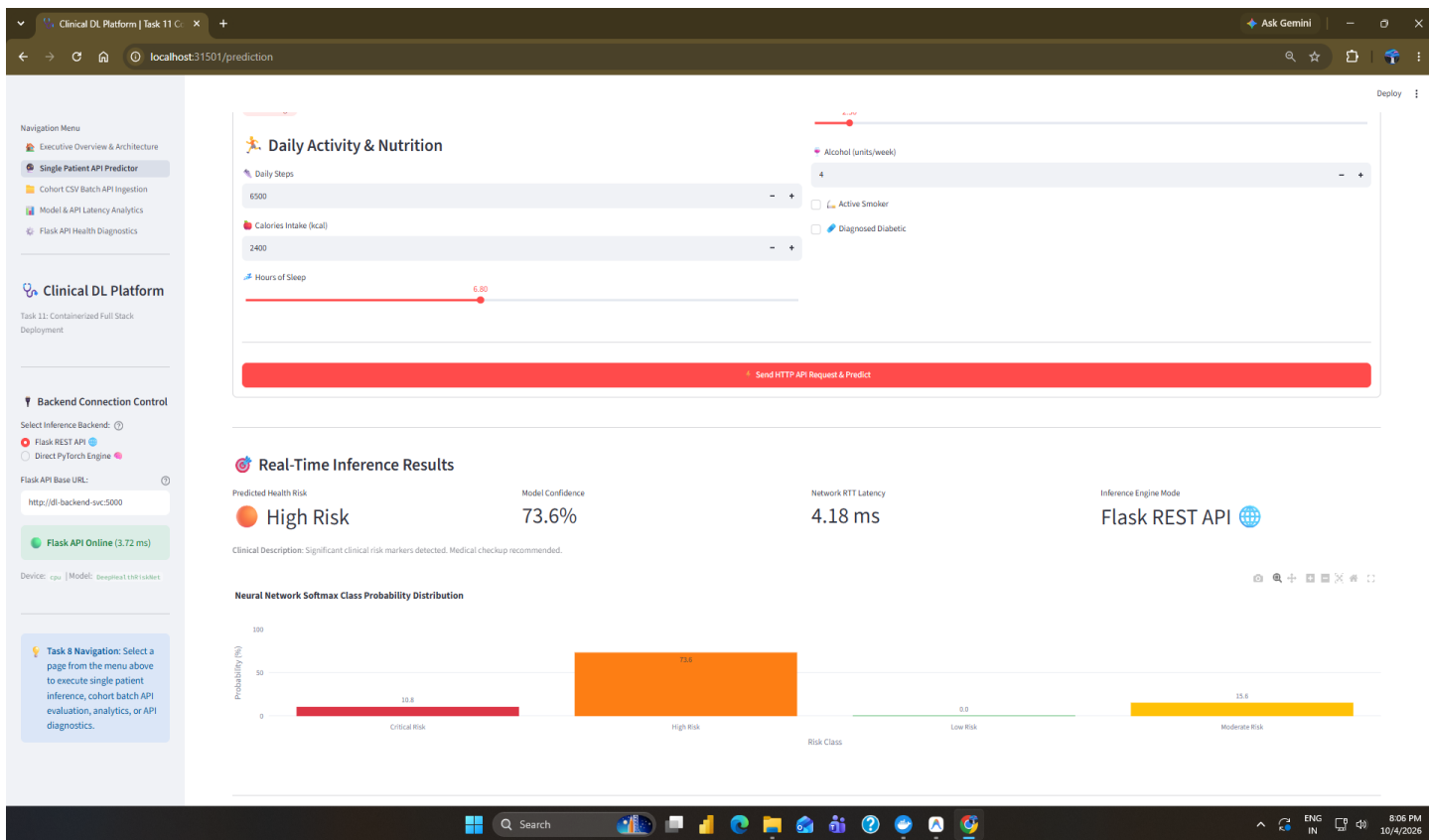


Figure 6: Clinician Streamlit Web Dashboard Running Live via NodePort (Port 31501)

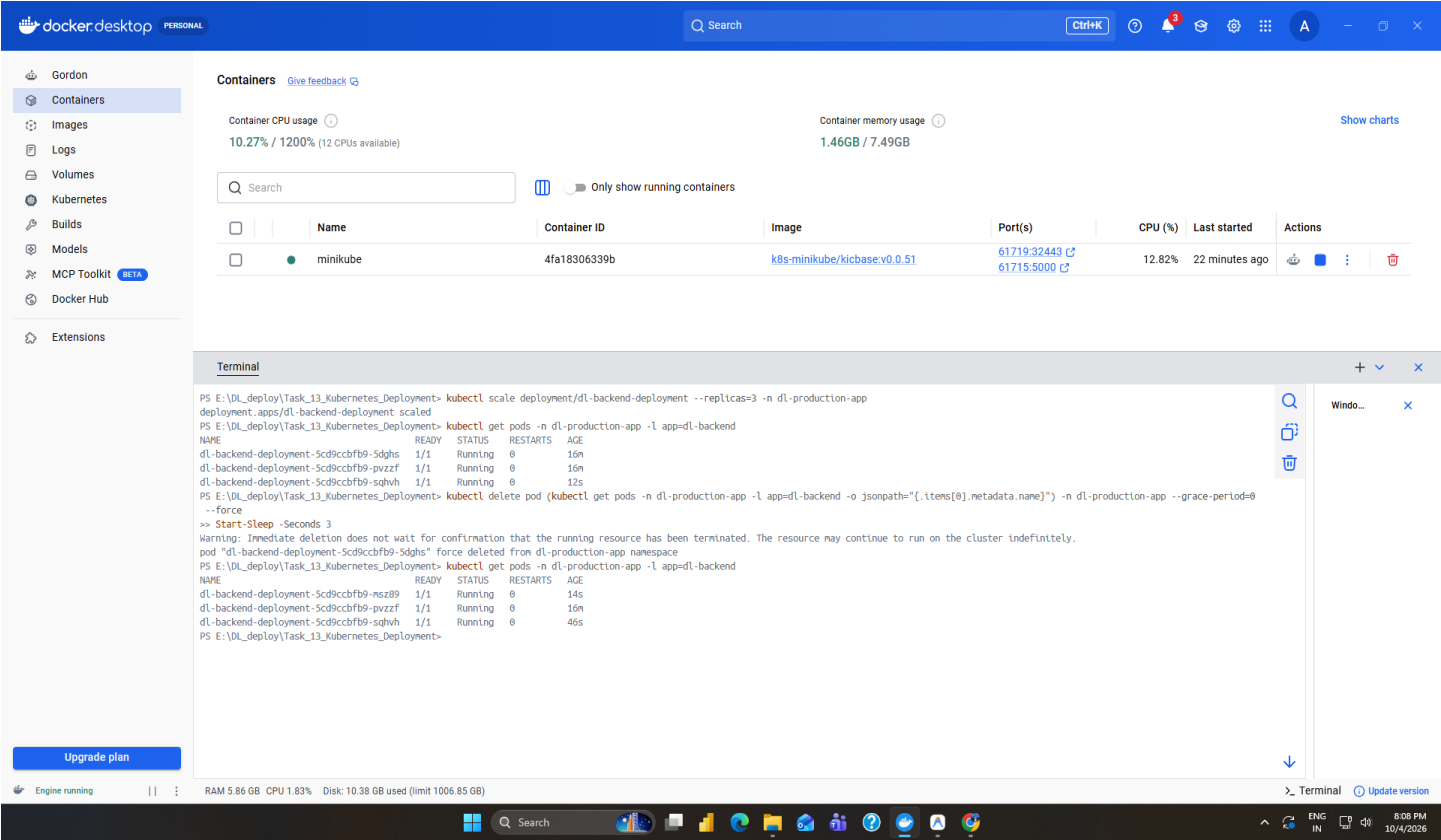


Figure 7: Dynamic Horizontal Pod Scaling & Automated Self-Healing Simulation

6. Technical Insights & Evaluation Compliance

- Evaluation Summary:
- **Successful Deployment:** Verified multi-replica deployment rollouts across backend and frontend microservices.
 - **Service Accessibility:** External access confirmed on both NodePort endpoints (Port 30500 for REST API, Port 31501 for UI).
 - **Configuration Quality:** Decoupled ConfigMap parameters, production compute limits (preventing OOM errors), and dual health probes.
 - **Fault Tolerance & Scaling:** Automated self-healing verified upon manual pod termination, maintaining desired 3-replica state.